

Software Architecture and Design Specification

Hostel Management System

Architecture description aligned with ISO/IEC/IEEE 42010:2022

Project	Hostel Management System
Document Version	1.0
Prepared for	Software Engineering Mini-Project

1. Introduction

1.1 Purpose

This document defines the architecture and detailed design of the Hostel Management System. It identifies the major architectural elements, their responsibilities, interfaces, security controls, traceability to requirements, and representative interaction sequences.

1.2 Scope

The architecture covers the functions defined in the SRS: authentication, student records, room records, room allocation/deallocation, occupancy information, authorization, and associated security behavior.

1.3 Architecture Principles

- Separate presentation, application-service, and data-access responsibilities.
- Enforce business rules in application services rather than relying only on the user interface.
- Use atomic database transactions for allocation changes.
- Apply security controls at the API/server boundary.
- Keep interfaces technology-neutral so the documented design can be implemented using the project-approved web framework.

2. Architecture Description

2.1 Stakeholders and Concerns

Stakeholder	Primary concern
Hostel Administrator / Warden	Correct student data, accurate room allocation, reliable occupancy information, secure administrative access.
Student	Confidentiality of own data and ability to view current allocation.
Development team	Maintainable separation of concerns and clear interfaces.
Reviewer / Tester	Traceability, verifiability, and reproducible behavior.

2.2 Architectural Pattern

The system uses a layered three-tier architecture consisting of a presentation/API layer, an application/service layer, and a data-access layer backed by a relational database. This pattern is selected to separate user-interface concerns, business rules, and persistence concerns; it also supports the maintainability requirement NFR-05.

2.3 Component Diagram

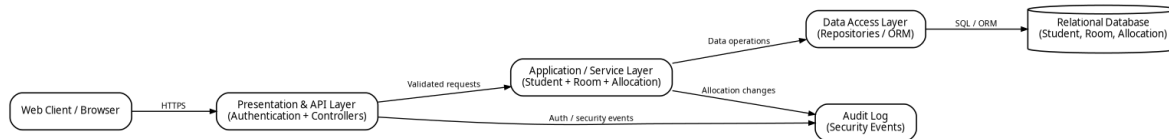


Fig. 1. High-level component view of the Hostel Management System.

2.4 Component Responsibilities

Component	Responsibilities
Web Client / Browser	Collects user input, displays results, and sends HTTPS requests. It shall not be the sole enforcement point for authorization rules.
Presentation & API Layer	Authenticates requests, validates request shape, maps API requests to services, and returns standardized responses.
Application / Service Layer	Implements student, room, and allocation business rules; coordinates transactions and security-relevant operations.
Data Access Layer	Encapsulates database access through repositories or ORM operations and provides persistence abstractions.
Relational Database	Stores student, room, and allocation state and provides transactional consistency.
Audit Log	Stores security-relevant event records needed by SEC-06.

2.5 Data Model

Entity	Key attributes	Relationships
Student	student_id, name, required hostel details	One student may have zero or one active allocation.
Room	room_id, capacity	One room may have multiple active occupants up to capacity.
Allocation	allocation_id, student_id, room_id, status, timestamps	Associates a student with a room and represents the allocation lifecycle.
User	user_id, role, password_hash, account status	Authenticates administrators/wardens and students.
AuditEvent	event_id, actor_id, event_type, timestamp, outcome	Records security-relevant actions.

2.6 Architecture Traceability

Requirements	Architectural elements	Traceability
FR-01	Presentation/API + Authentication	Authenticated access before protected functions.
FR-02, FR-03	Student Service + Student Repository	Create/update/search student records.
FR-04, FR-08	Room Service + Room Repository	Room capacity and occupancy.
FR-05, FR-06, FR-07	Allocation Service + Database transaction	Allocation rules and lifecycle.
FR-09	Authentication + Authorization middleware	Student can access only own allowed information.
NFR-03	Allocation Service + Database	Atomic allocation transaction.

Requirements	Architectural elements	Traceability
NFR-05	Layered architecture	Separation of concerns.
SEC-01 to SEC-06	API boundary + Auth + Audit Log + Database	Authentication, authorization, input protection, transport, credential, and logging controls.

3. Security Architecture

3.1 Security Boundaries

External browser traffic enters through the HTTPS/API boundary. Authentication and authorization are enforced by the application server before protected service operations are executed. Database access is restricted to the data-access layer. Direct browser access to the database is prohibited by the architecture.

3.2 Security Controls

- Authentication: credential verification occurs before protected operations.
- Authorization: role and resource ownership are checked before administrative or student-facing operations.
- Credential protection: password hashes are stored instead of plaintext passwords.
- Transport protection: deployed application traffic uses HTTPS/TLS.
- Input protection: request validation and parameterized/ORM database access are mandatory.
- Auditability: authentication and allocation events are recorded for security review.

4. Detailed Design

4.1 Sequence Diagram — Room Allocation

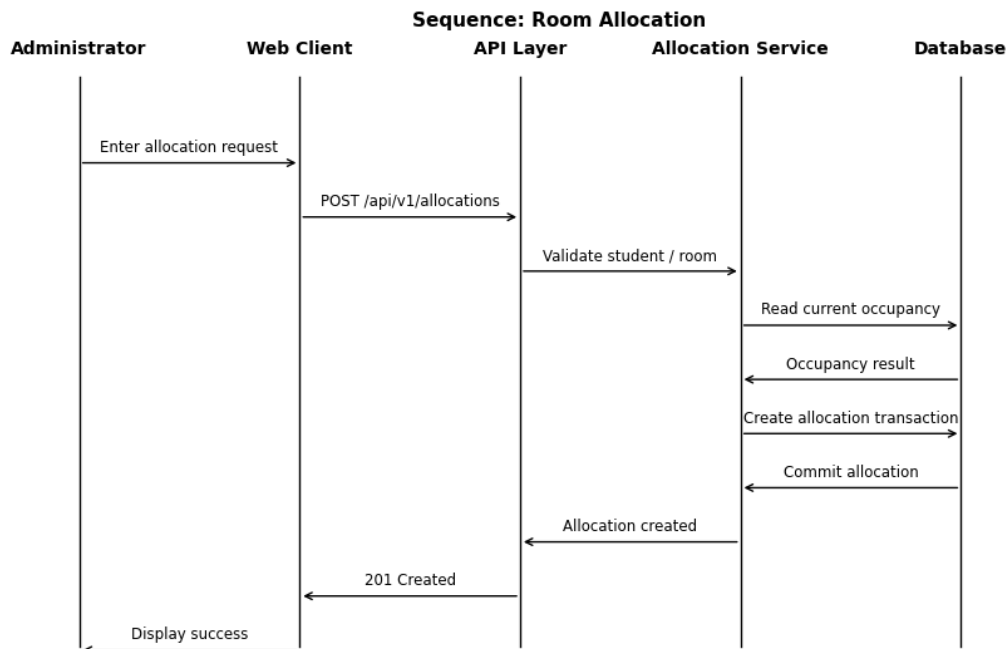


Fig. 2. Sequence for creating a room allocation.

4.2 Sequence Diagram — Student Details Update

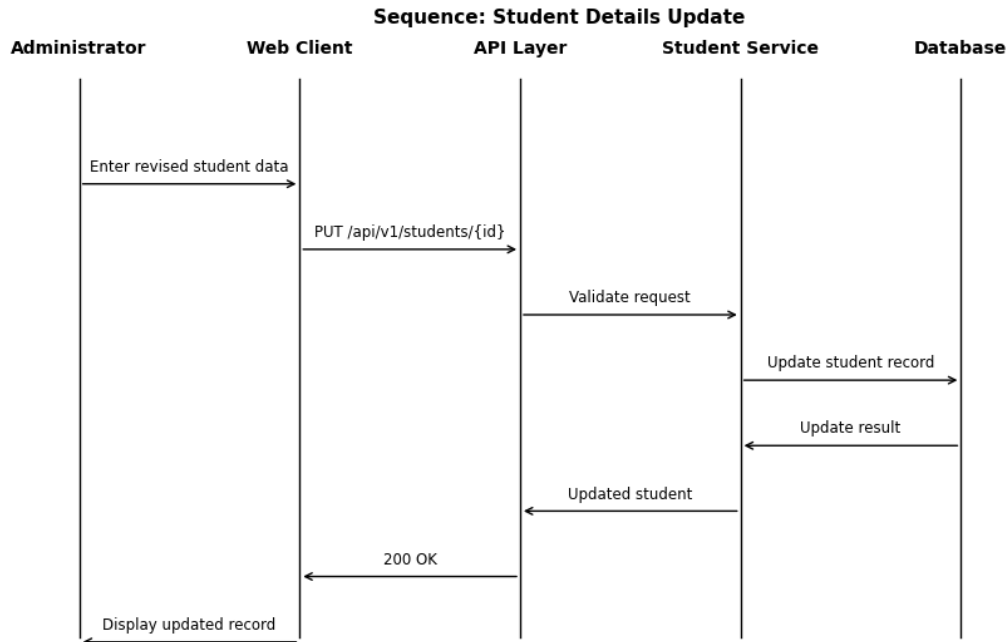


Fig. 3. Sequence for updating a student record.

4.3 API Design

The API is specified as a JSON-over-HTTPS interface. Resource identifiers are represented as path parameters. The exact web framework is implementation-specific.

Method and endpoint	Purpose	Authorization	Success
POST /api/v1/auth/login	Authenticate user.	Public endpoint with rate controls.	200 OK + session/token response.
GET /api/v1/students/{student_id}	Retrieve student record.	Administrator; student only for own record.	200 OK + student representation.
POST /api/v1/students	Create student record.	Administrator.	201 Created.
PUT /api/v1/students/{student_id}	Update student record.	Administrator.	200 OK.
GET /api/v1/rooms	List rooms and occupancy.	Authenticated user.	200 OK.
POST /api/v1/allocations	Create allocation.	Administrator.	201 Created.
PUT /api/v1/allocations/{allocation_id}	Modify allocation.	Administrator.	200 OK.
DELETE /api/v1/allocations/{allocation_id}	Deallocate room.	Administrator.	204 No Content.

4.4 API Error Handling

HTTP status	Condition	Required behavior
400 Bad Request	Malformed request structure.	Reject request and return machine-readable error details.
401 Unauthorized	No valid authentication.	Reject protected request without exposing protected data.
403 Forbidden	Authenticated user lacks permission.	Reject operation; no state change.

HTTP status	Condition	Required behavior
404 Not Found	Student, room, or allocation identifier does not exist.	Return resource-not-found error.
409 Conflict	Room full or student already allocated.	Return conflict error; preserve prior valid state.
422 Unprocessable Entity	Semantically invalid input.	Return field-level validation information.
500 Internal Server Error	Unexpected server failure.	Return generic error to client; record diagnostic information securely.

4.5 Standard Error Response

A JSON error response shall contain a stable application error code, a concise message, and optional field-level details. Sensitive implementation details, credentials, SQL statements, and stack traces shall not be returned to clients.

5. Design Decisions and Constraints

Item	Decision / constraint
Framework	Technology-neutral design; implementation may use an approved Python or PHP web framework.
Persistence	Relational database with transaction support is required by NFR-03.
Authorization	Server-side enforcement is mandatory; client-side hiding is insufficient.
Scalability target	The architecture is sized for the specified 50-concurrent-user performance test target, not an unverified claim of production capacity.
Out of scope	Fees, mess, visitors, attendance, complaints, and biometric access are not represented in this design.

6. Deployment View

A minimum deployment consists of a user browser, an HTTPS application server hosting the presentation/API and application-service layers, and a relational database server. The database is not directly exposed to client browsers. Security logs may be stored in the application database or a separate protected logging store, depending on deployment constraints.

References

- [1] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*, IEEE, 2018.
- [2] ISO/IEC/IEEE 29119-3:2021, *Software and systems engineering — Software testing — Part 3: Test documentation*, IEEE, 2021.
- [3] ISO/IEC/IEEE 42010:2022, *Software, systems and enterprise — Architecture description*, IEEE, 2022.
- [4] Software Engineering Mini-Project Deliverables Part-1, course handout, September 2026.